AIMLCZG546 — Software Engineering for Machine Learning

Assignment I — Requirements Formulation & System Architecture

Application: Loan Approval Classification System

Domain: Banking & Financial Services — Credit Risk / Loan Underwriting

Group No: 216

Submission Date: 06-July-2026

Group Member Details

Sl. No	BITS ID	Name	Contribution (Qualitative)	% Contribution
1	2025aa05444@wilp.bits-pilani.ac.in	PRASAD SHIVAJI KULKARNI	User Story and Requirements formulation & GR4ML modeling: problem statement, business/analytics/data-preparation views, and the top-3 quality requirements with justification. Testing and Validation of Application.	100
2	2025aa05387@wilp.bits-pilani.ac.in	SHELAR SACHIN KRISHNA	Data preparation pipeline: implemented and validated Filters 1-10 of the Pipe-and-Filter pipeline (loading, cleaning, outlier removal, feature/target split, train-val-test split, encoding, scaling) in train_loan_model.py.	100
3	2025aa05421@wilp.bits-pilani.ac.in	POWAR SAGAR GANPATI	System architecture & application: designed the CQRS + Pipe-and-Filter architecture diagram, built the Streamlit query-side app.py, and compiled the final report with screenshots and code appendices.	100
4	2025aa05326@wilp.bits-pilani.ac.in	SUJEET KUMAR YADAV	Model training & evaluation: implemented Filters 11-14 (Logistic Regression training, metric evaluation, artifact persistence), and produced the evaluation results / metrics comparison table.	100

1. Introduction & Problem Statement

Manual loan underwriting is slow, inconsistent across officers, and prone to human bias, while an over-lenient process increases the institution's exposure to default risk. This project formulates and implements a Machine-Learning-based Loan Approval Classification System for the Banking / Financial Services domain that predicts whether a loan application should be approved, based on the applicant's demographic profile, financial standing, and requested loan details.

Problem Statement: Given a set of applicant and loan-related attributes, predict the loan approval status (Approved / Not Approved) so that lending decisions can be made faster, more consistently, and with lower default risk than a purely manual process.

- Classification Type: Binary classification — 0 = Not Approved, 1 = Approved
- Target Variable: loan_status
- Business Context: Decision-support system for financial institutions' loan underwriting teams

2. Objective 1: Requirements Formulation

2.1 Requirement Specification & Measurable Goals

Functional Requirements:

- The system shall predict a binary loan approval outcome from 13 applicant/loan features.
- The system shall allow a user to upload a CSV of new applications and receive predictions in batch.
- The system shall display model performance metrics, a confusion matrix, and a classification report when ground-truth labels are available.
- The system shall expose the trained model, encoders, and scaler as versioned, reusable artifacts independent of the serving application.

Measurable (Non-functional) Goals:

- Achieve test-set Accuracy >= 0.85 and AUC-ROC >= 0.90 (achieved: 0.8927 / 0.9507).
- Keep training and serving responsibilities decoupled so that retraining never requires redeploying the serving application.
- Serve a prediction for a single applicant or an uploaded batch within an interactive (sub-second per row) response time in the Streamlit UI.

2.2 GR4ML Views

The application is modeled using the GR4ML (Goal-oriented Requirements for Machine Learning) notation across three complementary views: the Business View (why the system exists), the Analytics Design View (what analytics task delivers that value), and the Data Preparation View (how data is readied for that task).

2.2.1 Business View

The Business View captures the business actors, the top-level business goal and its AND-decomposed sub-goals, the KPIs that measure each sub-goal, and the business process the analytics capability supports.

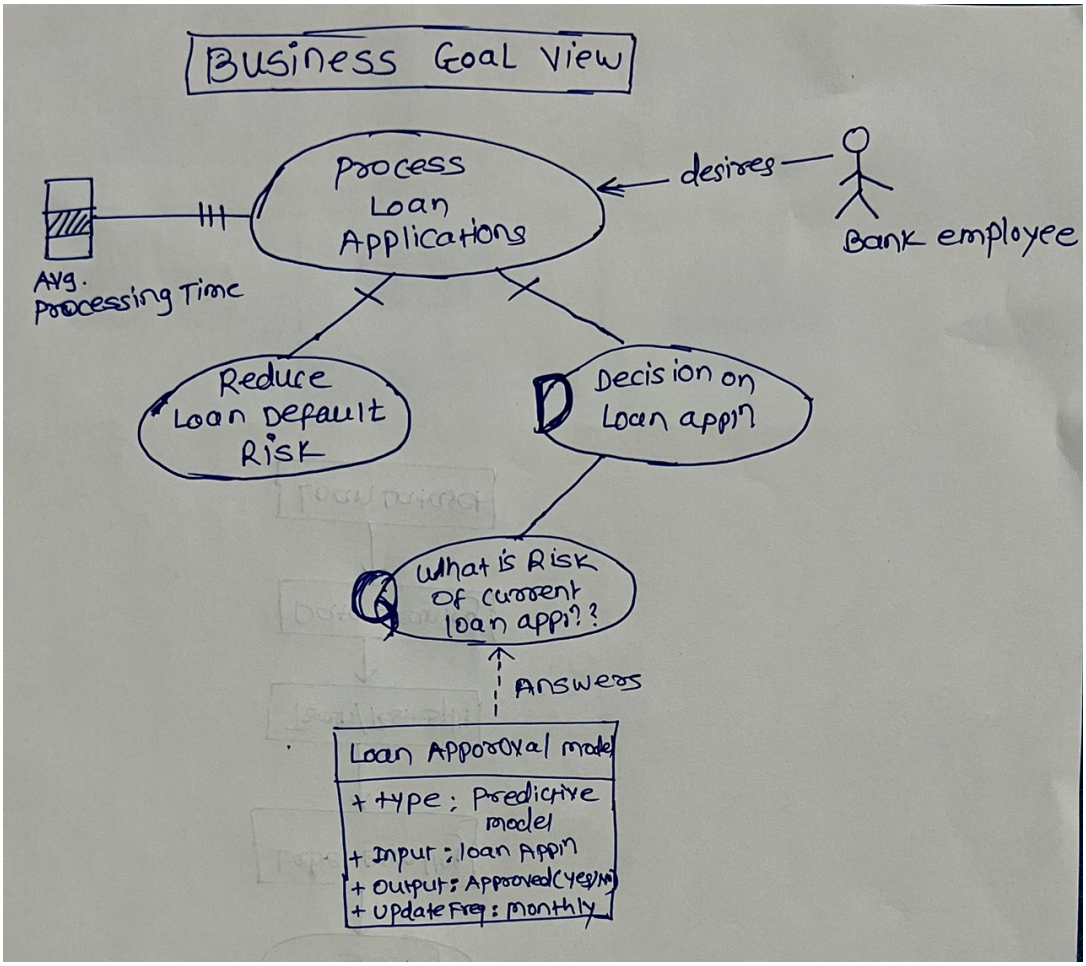


Figure 1: GR4ML Business View for the Loan Approval Classification System

The business goal decomposes into two AND-connected sub-goals — reducing loan default risk and enabling faster loan processing — both of which are satisfied by predicting loan status before approval, as summarized in the goal-decomposition tree below.

2.2.2 Analytics Design View

The Analytics Design View decomposes the Business View's need for analytics support into a concrete Analytics Goal, the Analysis Task used to achieve it, the technique selected, and the resulting Insight and Action, together with the quality targets the analytics goal must satisfy.

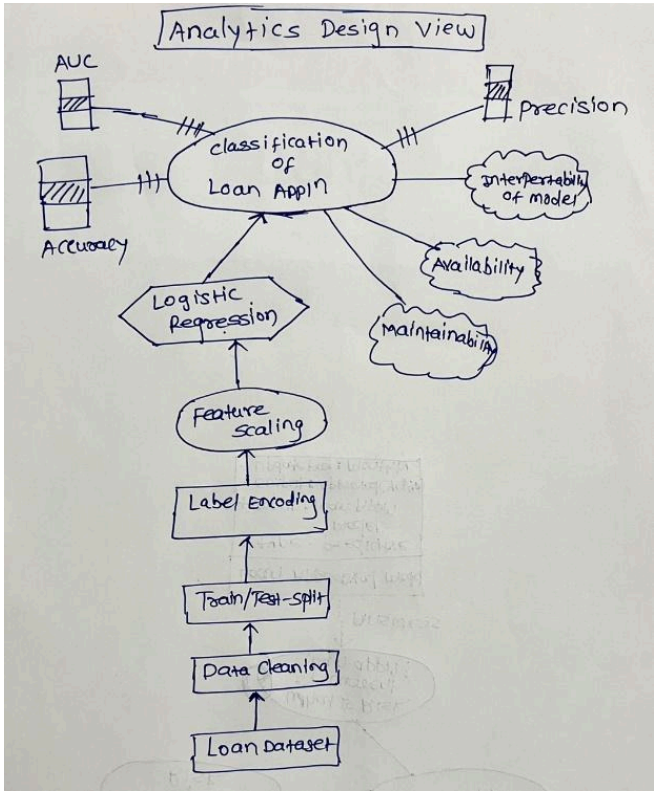


Figure 2: GR4ML Analytics Design View for the Loan Approval Classification System

2.2.3 Data Preparation View

The Data Preparation View traces the flow of data from the raw source through cleaning, feature engineering, and splitting, into the datasets that feed the Analysis Task. This directly mirrors the Pipe-and-Filter pipeline implemented in model/train_loan_model.py.

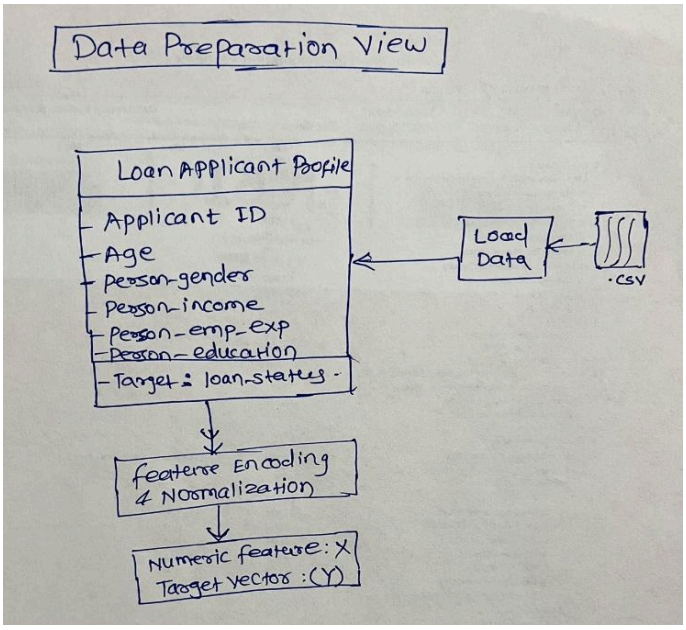


Figure 4: GR4ML Data Preparation View for the Loan Approval Classification System

Dataset Description:

- 45,000 raw loan application records; 44,995 after removing invalid age outliers.
- 13 input features spanning demographic, home-ownership, loan-detail, and financial categories, plus the target loan_status.
- Class distribution: 77.8% Not Approved (35,000) vs 22.2% Approved (10,000) — an imbalanced target, which is why AUC, F1, and MCC are reported alongside Accuracy.

Category	Feature	Type	Details
Demographic	person_age	Numeric	21-120 years (outliers removed)
Demographic	person_gender	Categorical	male / female
Demographic	person_education	Categorical	High School, Associate, Bachelor, Master, Doctorate
Demographic	person_income	Numeric	Annual income (~$12K-$600K)
Demographic	person_emp_exp	Numeric	Employment experience (years)
Home	person_home_ownership	Categorical	RENT, OWN, MORTGAGE, OTHER
Loan Details	loan_amnt	Numeric	Requested loan amount ($1K-$35K)
Loan Details	loan_intent	Categorical	PERSONAL, EDUCATION, MEDICAL, VENTURE, HOMEIMPROVEMENT, DEBTCONSOLIDATION
Loan Details	loan_int_rate	Numeric	Interest rate (%)
Financial	loan_percent_income	Numeric	Loan as % of income
Financial	cb_person_cred_hist_length	Numeric	Credit history length (years)
Financial	credit_score	Numeric	Credit score (500-789)
Financial	previous_loan_defaults_on_file	Categorical	Yes / No

2.3 Top Three Quality Requirements & Justification

The following quality requirements were prioritized for this application, based on the risk profile of automated lending decisions and the operational needs of the surrounding system.

#	Quality Requirement	Justification
1	Reliability / Predictive Accuracy	The model's decision directly affects loan applicants (financial access) and the institution (default risk). A low-accuracy or poorly-calibrated model would either reject creditworthy applicants or approve high-risk ones, causing direct financial and reputational loss. Measured via Accuracy (0.89), AUC (0.95) and MCC (0.69) on a held-out, stratified test set.
2	Maintainability	Credit policies, regulations, and applicant behaviour drift over time, so the model must be retrainable without touching the serving application. The CQRS separation (train_loan_model.py vs app.py) and the Pipe-and-Filter training pipeline (14 independent, single-responsibility filters) make the system easy to modify, test, and extend — e.g., swapping the model or adding a preprocessing step affects only one filter.
3	Scalability / Availability of the Serving Path	Loan officers and applicants query predictions far more often than the model is retrained. Because the Query side (Streamlit app) only reads immutable artifacts from the shared model store, it can be scaled or redeployed independently of the training pipeline with zero coupling, keeping the prediction service responsive under load.

3. Objective 2: System Architecture

3.1 System Architecture Diagram

The diagram below shows both the ML components (training pipeline, model artifacts, prediction execution) and the non-ML components (Streamlit UI, file-based model store, user interactions) of the system, and how the two architectural patterns described below compose together.

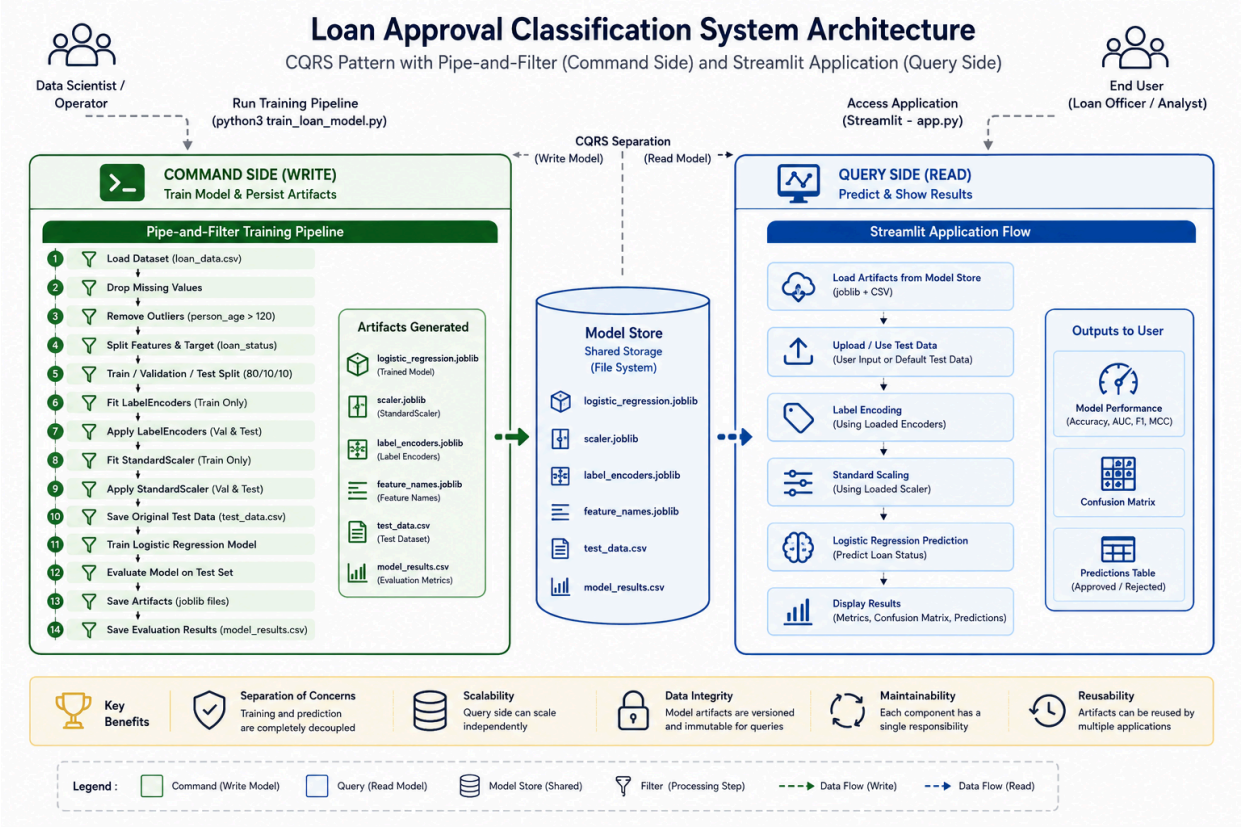


Figure 5: System Architecture — CQRS with a Pipe-and-Filter Command side and a Streamlit Query side

ML components: the 14-filter training pipeline, the fitted Logistic Regression model, the LabelEncoders and StandardScaler, and the prediction/evaluation logic in the Streamlit app.

Non-ML components: the Streamlit web UI (sidebar, file upload, charts), the file-system based Model Store that persists .joblib and .csv artifacts, and the CQRS boundary that routes requests between the Command and Query sides.

3.2 Architectural Patterns Applied

Pattern	Applied To	Why Chosen
Pipe-and-Filter	Command side — model/train_loan_model.py	The training workflow is naturally a sequence of independent transformation steps (load -> clean -> encode -> scale -> split -> train -> evaluate -> persist). Each step is implemented as a pure function (filter) whose return value (the pipe) feeds the next step, so steps can be tested, replaced or reordered independently.
CQRS (Command Query Responsibility Segregation)	Whole system — train_loan_model.py (Command) vs app.py (Query)	Training (write-heavy, run rarely) and prediction-serving (read-heavy, run on every user interaction) have very different responsibilities and scaling needs. Separating them into a Command side that writes model artifacts and a Query side that only reads those artifacts removes coupling, lets each side evolve and scale independently, and guarantees the serving app never mutates the model store.

3.3 Implementation of the Architectural Patterns

Pipe-and-Filter (model/train_loan_model.py): each preprocessing/training step is a standalone function; the pipeline's orchestrator, run_pipeline(), wires the filters together purely through function return values (the pipes):

```
def run_pipeline(data_path='loan_data.csv'):
    raw_df = load_data(data_path)                    # FILTER 1
    clean_df = drop_missing(raw_df)                  # FILTER 2
    filtered_df = remove_outliers(clean_df)          # FILTER 3
    X, y, cat_cols, num_cols = \
        split_features_target(filtered_df)           # FILTER 4
    X_train, X_val, X_test, y_train, y_val, y_test = \
        split_datasets(X, y)                          # FILTER 5
    X_train_enc, encoders = \
        fit_label_encoders(X_train, cat_cols)         # FILTER 6
    X_val_enc, X_test_enc = apply_label_encoders(
        X_val, X_test, encoders, cat_cols)            # FILTER 7
    X_train_scaled, scaler = fit_scaler(X_train_enc)  # FILTER 8
    X_val_scaled, X_test_scaled = apply_scaler(
        X_val_enc, X_test_enc, scaler, feature_names) # FILTER 9
    save_test_data(X_test, y_test)                   # FILTER 10
    model = train_model(X_train_scaled, y_train)     # FILTER 11
    metrics = evaluate_model(model, X_test_scaled, y_test)  # FILTER 12
    save_artifacts(model, scaler, encoders, feature_names)  # FILTER 13
    save_results([metrics])                          # FILTER 14
```

CQRS (train_loan_model.py = Command, app.py = Query): the Command side writes model artifacts to disk and never reads them back; the Query side (Streamlit app) only reads those artifacts and never mutates them — the file system is the single boundary between the two sides:

```
# QUERY SIDE (app.py) — read-only access to the model store
@st.cache_resource
def load_models():
    return {'Logistic Regression':
            joblib.load('model/logistic_regression.joblib')}

@st.cache_resource
def load_preprocessors():
    scaler = joblib.load('model/scaler.joblib')
    label_encoders = joblib.load('model/label_encoders.joblib')
    feature_names = joblib.load('model/feature_names.joblib')
    return scaler, label_encoders, feature_names
```

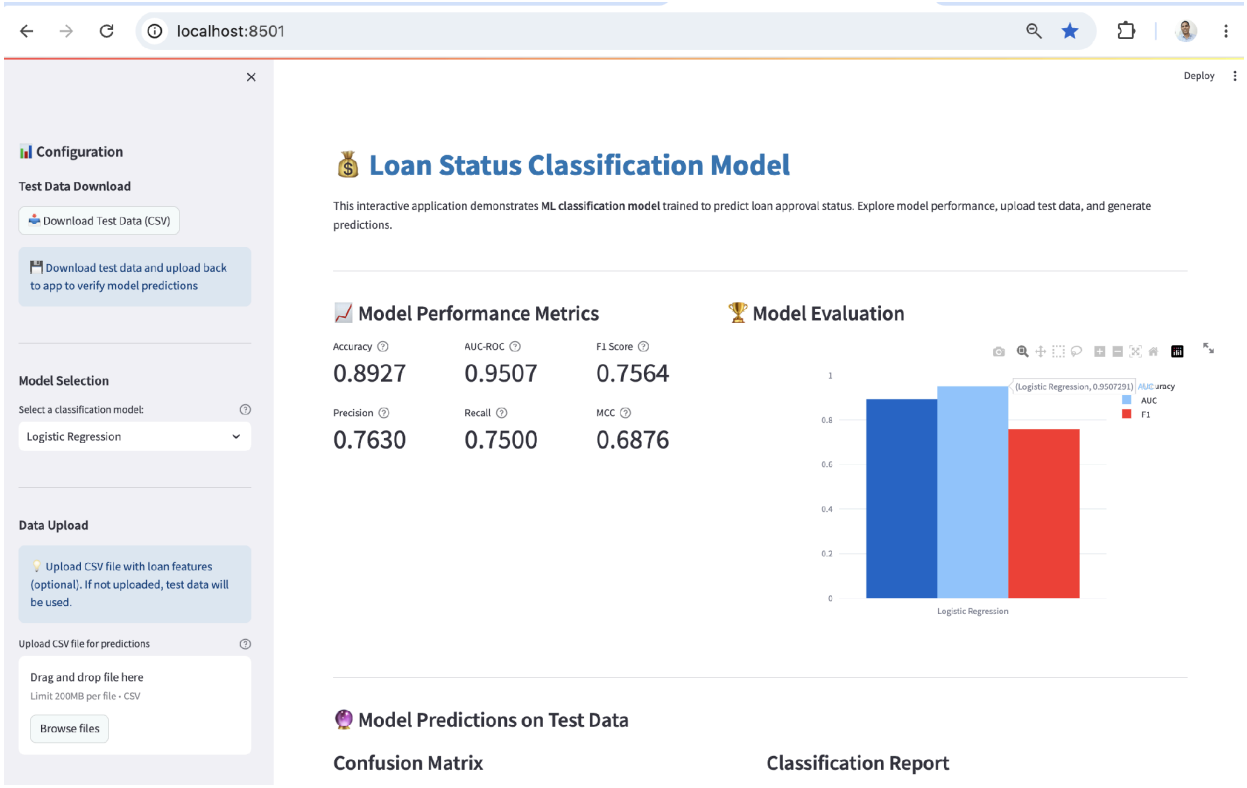
```
# Query execution — stateless, read-only prediction
y_pred = model.predict(data_preprocessed)
```

3.4 Technology Stack

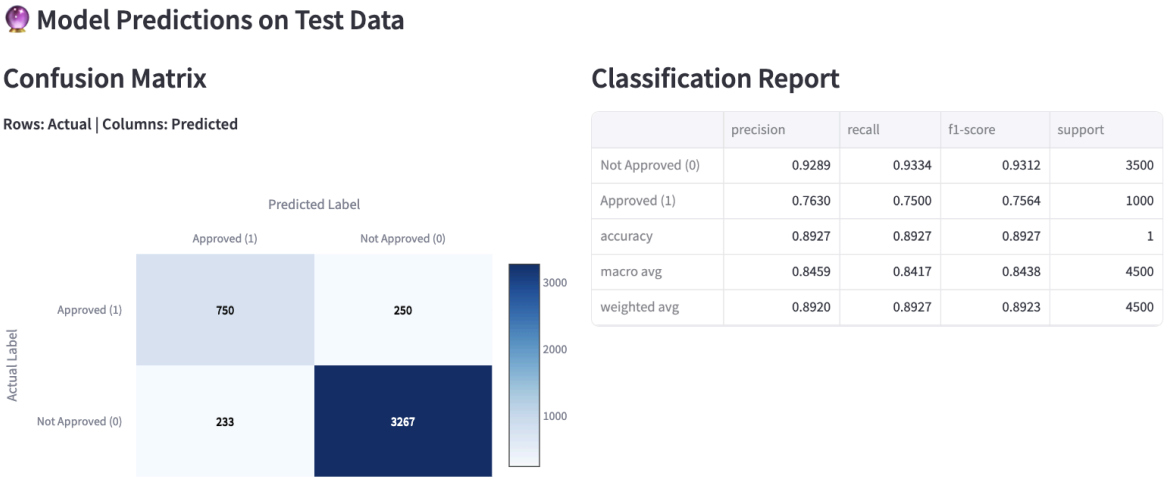
- Language: Python 3
- Modeling: scikit-learn (LogisticRegression, StandardScaler, LabelEncoder)
- Serving / UI: Streamlit, Plotly (charts, confusion-matrix heatmap)
- Data handling: pandas, numpy
- Artifact persistence (Model Store): joblib, CSV

4. Application Screenshots & Explanation

Screenshot 1 — Model Performance Dashboard: the landing view of the Streamlit app (Query side). The sidebar lets the user download test data, choose a model, and upload a CSV of new applications. The main panel reports Accuracy (0.8927), AUC-ROC (0.9507), Precision, Recall, F1, and MCC for the selected model, alongside a comparison bar chart.

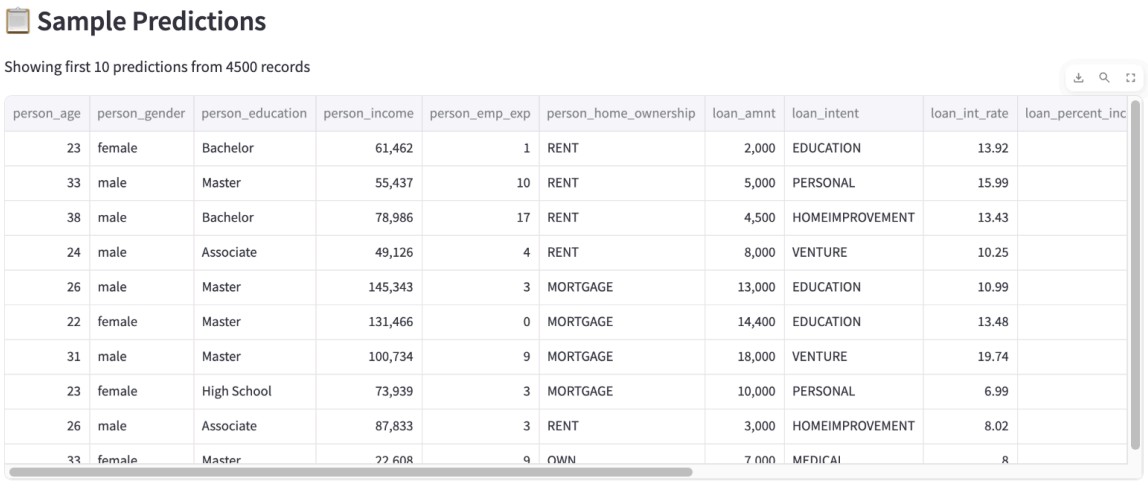


Screenshot 2 — Confusion Matrix & Classification Report: generated once ground-truth labels are available (default test set or an uploaded CSV containing loan_status). Of 1,000 actually-approved applicants, 750 were correctly predicted Approved (true positives) and 250 were missed (false negatives); of 3,500 actually-rejected applicants, 3,267 were correctly predicted Not Approved. The classification report breaks precision/recall/F1 out per class and confirms the weighted accuracy of 0.8927.



	precision	recall	f1-score	support
Not Approved (0)	0.9289	0.9334	0.9312	3500
Approved (1)	0.7630	0.7500	0.7564	1000
accuracy	0.8927	0.8927	0.8927	1
macro avg	0.8459	0.8417	0.8438	4500
weighted avg	0.8920	0.8927	0.8923	4500

Screenshot 3 — Sample Predictions Table: the first 10 rows of the batch being scored, showing the original (pre-encoding) feature values together with the model's Approved/Not Approved prediction, so a loan officer can trace a decision back to the applicant's raw attributes.



📋 Sample Predictions

Showing first 10 predictions from 4500 records

person_age	person_gender	person_education	person_income	person_emp_exp	person_home_ownership	loan_amnt	loan_intent	loan_int_rate	loan_percent_inc
23	female	Bachelor	61,462	1	RENT	2,000	EDUCATION	13.92	
33	male	Master	55,437	10	RENT	5,000	PERSONAL	15.99	
38	male	Bachelor	78,986	17	RENT	4,500	HOMEIMPROVEMENT	13.43	
24	male	Associate	49,126	4	RENT	8,000	VENTURE	10.25	
26	male	Master	145,343	3	MORTGAGE	13,000	EDUCATION	10.99	
22	female	Master	131,466	0	MORTGAGE	14,400	EDUCATION	13.48	
31	male	Master	100,734	9	MORTGAGE	18,000	VENTURE	19.74	
23	female	High School	73,939	3	MORTGAGE	10,000	PERSONAL	6.99	
26	male	Associate	87,833	3	RENT	3,000	HOMEIMPROVEMENT	8.02	
33	female	Master	22,608	9	OWN	7,000	MEDICAL	8	

5. Evaluation Results

The Logistic Regression baseline was evaluated on a held-out, stratified test set of 4,500 applications:

Metric	Value	Definition
Accuracy	0.8927	Overall proportion of correct predictions
AUC-ROC	0.9507	Ability of the model to discriminate between classes
Precision	0.7630	True positives / all predicted approvals
Recall	0.7500	True positives / all actual approvals
F1 Score	0.7564	Harmonic mean of precision and recall
MCC	0.6876	Matthews Correlation Coefficient; robust for imbalanced data

6. Repository Structure & How to Run

```
project-folder/
├── app.py                  # Streamlit web app (CQRS Query side)
├── requirements.txt        # Python dependencies
├── README.md
├── system_architecture.png
└── model/
    ├── train_loan_model.py       # Training pipeline
    │                             # (CQRS Command side, Pipe-and-Filter)
    ├── loan_data.csv              # Original dataset
    ├── logistic_regression.joblib # Trained model
    ├── scaler.joblib               # StandardScaler
    ├── label_encoders.joblib       # Categorical encoders
    ├── feature_names.joblib
    ├── test_data.csv               # Test dataset (original values)
    └── model_results.csv           # Evaluation metrics
```

```
# Command – train the model
cd model/ && python3 train_loan_model.py

# Query – run the Streamlit application
pip install -r requirements.txt
streamlit run app.py
```

7. Group Member Contribution

The contribution of each group member in executing this assignment is summarized below (see also the Group Member Details table on the title page).

Sl. No	BITS ID	Name	Contribution (Qualitative)	% Contribution
1	2025aa05444@wilp.bits-pilani.ac.in	PRASAD SHIVAJI KULKARNI	User Story and Requirements formulation & GR4ML modeling: problem statement, business/analytics/data-preparation views, and the top-3 quality requirements with justification. Testing and Validation of Application.	100
2	2025aa05387@wilp.bits-pilani.ac.in	SHELAR SACHIN KRISHNA	Data preparation pipeline: implemented and validated Filters 1-10 of the Pipe-and-Filter pipeline (loading, cleaning, outlier removal, feature/target split, train-val-test split, encoding, scaling) in train_loan_model.py.	100
3	2025aa05421@wilp.bits-pilani.ac.in	POWAR SAGAR GANPATI	System architecture & application: designed the CQRS + Pipe-and-Filter architecture diagram, built the Streamlit query-side app.py, and compiled the final report with screenshots and code appendices.	100
4	2025aa05326@wilp.bits-pilani.ac.in	SUJEET KUMAR YADAV	Model training & evaluation: implemented Filters 11-14 (Logistic Regression training, metric evaluation, artifact persistence), and produced the evaluation results / metrics comparison table.	100

8. Conclusion

This assignment modeled a Loan Approval Classification System end-to-end using GR4ML's Business, Analytics Design, and Data Preparation views, identified Reliability, Maintainability, and Scalability as the top quality requirements, and implemented a working system architecture combining the Pipe-and-Filter and CQRS patterns. The resulting Logistic Regression model achieves 0.89 accuracy and 0.95 AUC on held-out data, and is served through a Streamlit application that is fully decoupled from the training pipeline.

9. Appendix A — Full Source Code

Full, unabridged source code of the two files implementing the architectural patterns described in Section 3, included per the submission guidelines.

9.1 model/train_loan_model.py — Command Side (Pipe-and-Filter Training Pipeline)

```
"""
╔═══════════════════════════════════════════════════════════════════════╗
║          CQRS PATTERN — COMMAND SIDE                                  ║
║  Responsibility : WRITE / mutate the model store                      ║
║  Triggers       : Run manually (python3 train_loan_model.py)          ║
║  Output         : *.joblib artifacts + CSV reports                    ║
╚═══════════════════════════════════════════════════════════════════════╝

CQRS Separation
---------------
COMMAND (this file)          |  QUERY (app.py)
─────────────────────────────┼──────────────────────────────────────────
Trains the model             |  Loads trained artifacts
Encodes & scales features    |  Encodes & scales user input
Writes *.joblib to disk      |  Reads *.joblib from disk
Writes CSV reports           |  Reads CSV reports
No HTTP / UI concerns        |  No training logic
Run once (or on demand)      |  Runs on every user request

Internally this file also demonstrates the Pipe and Filter pattern:
    Each function is a FILTER; its return value is the PIPE flowing into the next filter.

Pipeline flow:
    raw CSV path
        |-- [FILTER 1: load_data]             --> raw DataFrame
        |-- [FILTER 2: drop_missing]          --> cleaned DataFrame
        |-- [FILTER 3: remove_outliers]       --> outlier-free DataFrame
        |-- [FILTER 4: split_features_target] --> (X, y, cat_cols, num_cols)
        |-- [FILTER 5: split_datasets]        --> train / val / test splits
        |-- [FILTER 6: fit_label_encoders]    --> (X_train_encoded, encoders)
        |-- [FILTER 7: apply_label_encoders]  --> (X_val_encoded, X_test_encoded)
        |-- [FILTER 8: fit_scaler]            --> (X_train_scaled, scaler)
        |-- [FILTER 9: apply_scaler]          --> (X_val_scaled, X_test_scaled)
        |-- [FILTER 10: save_test_data]       --> test_data.csv  (side-effect)
        |-- [FILTER 11: train_model]          --> trained model
        |-- [FILTER 12: evaluate_model]       --> metrics dict
        |-- [FILTER 13: save_artifacts]       --> *.joblib files (side-effect)
        |-- [FILTER 14: save_results]         --> model_results.csv (side-effect)
"""

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, roc_auc_score, precision_score, recall_score,
    f1_score, matthews_corrcoef
)
import joblib
import warnings
warnings.filterwarnings('ignore')


# ============================================================
# FILTER 1 — Load raw data from disk
# PIPE IN  : file path (str)
# PIPE OUT : raw DataFrame
# ============================================================
def load_data(filepath: str) -> pd.DataFrame:

    print(f"[FILTER 1] Loading dataset from '{filepath}' ...")
    df = pd.read_csv(filepath)
    print(f"           Shape: {df.shape}")
    print(f"           Target distribution:\n{df['loan_status'].value_counts()}")
    return df
```

```
# ============================================================
# FILTER 2 — Drop rows with missing values
# PIPE IN  : raw DataFrame
# PIPE OUT : DataFrame without NaN rows
# ============================================================
def drop_missing(df: pd.DataFrame) -> pd.DataFrame:
    print("\n[FILTER 2] Dropping missing values ...")
   #print(f"           Missing counts before:\n{df.isnull().sum()}")
    df_clean = df.dropna()
    print(f"           Shape after drop: {df_clean.shape}")
    return df_clean


# ============================================================
# FILTER 3 — Remove obvious outliers (age > 120)
# PIPE IN  : cleaned DataFrame
# PIPE OUT : outlier-free DataFrame
# ============================================================
def remove_outliers(df: pd.DataFrame) -> pd.DataFrame:
    print("\n[FILTER 3] Removing age outliers (person_age > 120) ...")
    df_filtered = df[df['person_age'] <= 120]
    print(f"           Shape after outlier removal: {df_filtered.shape}")
    return df_filtered


# ============================================================
# FILTER 4 — Separate features (X) from target (y)
# PIPE IN  : clean DataFrame
# PIPE OUT : (X, y, categorical_cols, numerical_cols)
# ============================================================
def split_features_target(df: pd.DataFrame, target_col: str = 'loan_status'):
    print(f"\n[FILTER 4] Splitting features and target ('{target_col}') ...")
    X = df.drop(target_col, axis=1)
    y = df[target_col]
    categorical_cols = X.select_dtypes(include=['object']).columns.tolist()
    numerical_cols   = X.select_dtypes(include=['float64', 'int64']).columns.tolist()
    print(f"           Categorical columns : {categorical_cols}")
    print(f"           Numerical columns   : {numerical_cols}")
    return X, y, categorical_cols, numerical_cols


# ============================================================
# FILTER 5 — Stratified train / validation / test split
# PIPE IN  : (X, y)
# PIPE OUT : (X_train_orig, X_val_orig, X_test_orig,
#             y_train, y_val, y_test)
# ============================================================
def split_datasets(X: pd.DataFrame, y: pd.Series):
    print("\n[FILTER 5] Splitting into train / val / test sets ...")
    X_temp, X_test_orig, y_temp, y_test = train_test_split(
        X, y, test_size=0.1, random_state=42, stratify=y
    )
    X_train_orig, X_val_orig, y_train, y_val = train_test_split(

        X_temp, y_temp, test_size=0.111, random_state=42, stratify=y_temp
    )
    print(f"           Train : {X_train_orig.shape[0]} rows")
    print(f"           Val   : {X_val_orig.shape[0]} rows")
    print(f"           Test  : {X_test_orig.shape[0]} rows")
    return X_train_orig, X_val_orig, X_test_orig, y_train, y_val, y_test


# ============================================================
# FILTER 6 — Fit LabelEncoders on TRAINING data only
# PIPE IN  : (X_train_orig, categorical_cols)
# PIPE OUT : (X_train_encoded DataFrame, label_encoders dict)
# ============================================================
def fit_label_encoders(X_train_orig: pd.DataFrame, categorical_cols: list):
    print("\n[FILTER 6] Fitting LabelEncoders on training data only ...")
    label_encoders = {}
    X_train_encoded = X_train_orig.copy()
    for col in categorical_cols:
        le = LabelEncoder()
        X_train_encoded[col] = le.fit_transform(X_train_orig[col].astype(str))
        label_encoders[col] = le
        print(f"           Encoded '{col}': {dict(zip(le.classes_, le.transform(le.classes_)))}")
    return X_train_encoded, label_encoders


# ============================================================
# FILTER 7 — Apply fitted encoders to validation and test sets
# PIPE IN  : (X_val_orig, X_test_orig, label_encoders, categorical_cols)
# PIPE OUT : (X_val_encoded, X_test_encoded)
```

```
# ============================================================
def apply_label_encoders(X_val_orig: pd.DataFrame, X_test_orig: pd.DataFrame,
                         label_encoders: dict, categorical_cols: list):
    print("\n[FILTER 7] Applying LabelEncoders to val and test sets ...")
    X_val_encoded  = X_val_orig.copy()
    X_test_encoded = X_test_orig.copy()
    for col in categorical_cols:
        X_val_encoded[col]  = label_encoders[col].transform(X_val_orig[col].astype(str))
        X_test_encoded[col] = label_encoders[col].transform(X_test_orig[col].astype(str))
    print("           Encoding applied.")
    return X_val_encoded, X_test_encoded


# ============================================================
# FILTER 8 — Fit StandardScaler on TRAINING data only
# PIPE IN  : X_train_encoded DataFrame
# PIPE OUT : (X_train_scaled DataFrame, fitted scaler)
# ============================================================
def fit_scaler(X_train_encoded: pd.DataFrame):
    print("\n[FILTER 8] Fitting StandardScaler on training data only ...")
    feature_names = list(X_train_encoded.columns)
    scaler = StandardScaler()
    X_train_scaled = pd.DataFrame(
        scaler.fit_transform(X_train_encoded),
        columns=feature_names
    )
    print("           Scaler fitted.")
    return X_train_scaled, scaler


# ============================================================

# FILTER 9 — Apply fitted scaler to validation and test sets
# PIPE IN  : (X_val_encoded, X_test_encoded, scaler, feature_names)
# PIPE OUT : (X_val_scaled, X_test_scaled)  — both DataFrames
# ============================================================
def apply_scaler(X_val_encoded: pd.DataFrame, X_test_encoded: pd.DataFrame,
                 scaler: StandardScaler, feature_names: list):
    print("\n[FILTER 9] Applying StandardScaler to val and test sets ...")
    X_val_scaled  = pd.DataFrame(scaler.transform(X_val_encoded),  columns=feature_names)
    X_test_scaled = pd.DataFrame(scaler.transform(X_test_encoded), columns=feature_names)
    print("           Scaling applied.")
    return X_val_scaled, X_test_scaled


# ============================================================
# FILTER 10 — Persist test set with original categorical values
# PIPE IN  : (X_test_orig, y_test)
# PIPE OUT : path to saved CSV (side-effect: writes file)
# ============================================================
def save_test_data(X_test_orig: pd.DataFrame, y_test: pd.Series,
                   output_path: str = 'test_data.csv') -> str:
    print(f"\n[FILTER 10] Saving original test data to '{output_path}' ...")
    test_data = X_test_orig.copy()
    test_data['loan_status'] = y_test.values
    test_data.to_csv(output_path, index=False)
    print(f"            Saved {len(test_data)} rows.")
    return output_path


# ============================================================
# FILTER 11 — Train Logistic Regression model
# PIPE IN  : (X_train_scaled, y_train)
# PIPE OUT : fitted LogisticRegression model
# ============================================================
def train_model(X_train: pd.DataFrame, y_train: pd.Series) -> LogisticRegression:
    print("\n[FILTER 11] Training Logistic Regression model ...")
    model = LogisticRegression(max_iter=1000, random_state=42, solver='lbfgs')
    model.fit(X_train, y_train)
    print("            Model training complete.")
    return model


# ============================================================
# FILTER 12 — Evaluate model on test set
# PIPE IN  : (model, X_test_scaled, y_test)
# PIPE OUT : metrics dict
# ============================================================
def evaluate_model(model: LogisticRegression,
                   X_test: pd.DataFrame, y_test: pd.Series) -> dict:
    print("\n[FILTER 12] Evaluating model on test set ...")
    y_pred = model.predict(X_test)
    metrics = {
```

```
        'Model'    : 'Logistic Regression',
        'Accuracy' : accuracy_score(y_test, y_pred),
        'AUC'      : roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]),
        'Precision': precision_score(y_test, y_pred),
        'Recall'   : recall_score(y_test, y_pred),
        'F1'       : f1_score(y_test, y_pred),
        'MCC'      : matthews_corrcoef(y_test, y_pred),
    }
    print(f"            Accuracy  : {metrics['Accuracy']:.4f}")

    print(f"            AUC       : {metrics['AUC']:.4f}")
    print(f"            Precision : {metrics['Precision']:.4f}")
    print(f"            Recall    : {metrics['Recall']:.4f}")
    print(f"            F1 Score  : {metrics['F1']:.4f}")
    print(f"            MCC       : {metrics['MCC']:.4f}")
    return metrics


# ============================================================
# FILTER 13 — Persist model + preprocessor artifacts to disk
# PIPE IN  : (model, scaler, label_encoders, feature_names)
# PIPE OUT : list of saved file paths (side-effect: writes files)
# ============================================================
def save_artifacts(model: LogisticRegression, scaler: StandardScaler,
                   label_encoders: dict, feature_names: list) -> list:
    print("\n[FILTER 13] Saving model and preprocessor artifacts ...")
    paths = {
        'logistic_regression.joblib': model,
        'scaler.joblib'             : scaler,
        'label_encoders.joblib'     : label_encoders,
        'feature_names.joblib'      : feature_names,
    }
    for filename, obj in paths.items():
        joblib.dump(obj, filename)
        print(f"            Saved: {filename}")
    return list(paths.keys())


# ============================================================
# FILTER 14 — Save evaluation results to CSV
# PIPE IN  : list of metrics dicts
# PIPE OUT : path to saved results CSV (side-effect: writes file)
# ============================================================
def save_results(results: list, output_path: str = 'model_results.csv') -> str:
    print(f"\n[FILTER 14] Saving evaluation results to '{output_path}' ...")
    results_df = pd.DataFrame(results)
    results_df.to_csv(output_path, index=False)
    print("\n" + "="*60)
    print("SUMMARY RESULTS")
    print("="*60)
    print(results_df.to_string(index=False))
    return output_path


# ============================================================
# CQRS COMMAND HANDLER
# run_pipeline() is the single entry-point for the Command side.
# It orchestrates all filters and WRITES state to the model store
# (*.joblib files + CSV reports). The Query side (app.py) will
# later READ from that store — the two sides never share memory.
# ============================================================
def run_pipeline(data_path: str = 'loan_data.csv'):
    print("=" * 60)
    print("LOAN APPROVAL : CQRS ARCHITECTURE - COMMAND SIDE")
    print("=" * 60)
    print("Command Handler : run_pipeline()")
    print("Responsibility  : Train model and persist artifacts")
    print("Pattern         : Pipe-and-Filter")
    print("Pipeline        : Each filter processes data and passes")
    print("                  the output to the next filter.")

    print("=" * 60)
    print("\nStarting Loan Approval Training Pipeline...\n")

    # --- PIPE: file path ----------------------------------
    raw_df = load_data(data_path)                           # FILTER 1

    # --- PIPE: raw DataFrame ------------------------------
    clean_df = drop_missing(raw_df)                         # FILTER 2

    # --- PIPE: NaN-free DataFrame -------------------------
    filtered_df = remove_outliers(clean_df)                 # FILTER 3
```

```
    # --- PIPE: outlier-free DataFrame ----------------------
    X, y, cat_cols, num_cols = split_features_target(         # FILTER 4
        filtered_df, target_col='loan_status'
    )

    # --- PIPE: (X, y, col lists) --------------------------
    X_train_orig, X_val_orig, X_test_orig, \
    y_train, y_val, y_test = split_datasets(X, y)             # FILTER 5

    # --- PIPE: original split DataFrames + Series ----------
    X_train_encoded, label_encoders = fit_label_encoders(     # FILTER 6
        X_train_orig, cat_cols
    )

    # --- PIPE: encoded train + fitted encoders -------------
    X_val_encoded, X_test_encoded = apply_label_encoders(     # FILTER 7
        X_val_orig, X_test_orig, label_encoders, cat_cols
    )

    # --- PIPE: encoded val & test DataFrames ---------------
    X_train_scaled, scaler = fit_scaler(X_train_encoded)      # FILTER 8

    # --- PIPE: scaled train DataFrame + fitted scaler ------
    feature_names = list(X.columns)
    X_val_scaled, X_test_scaled = apply_scaler(               # FILTER 9
        X_val_encoded, X_test_encoded, scaler, feature_names
    )

    # --- PIPE: scaled val & test DataFrames ----------------
    save_test_data(X_test_orig, y_test)                       # FILTER 10

    # --- PIPE: (X_train_scaled, y_train) -------------------
    model = train_model(X_train_scaled, y_train)              # FILTER 11

    # --- PIPE: trained model + (X_test_scaled, y_test) -----
    metrics = evaluate_model(model, X_test_scaled, y_test)    # FILTER 12

    # --- PIPE: metrics dict --------------------------------
    save_artifacts(model, scaler, label_encoders,             # FILTER 13
                   feature_names)

    # --- PIPE: list of metrics dicts -----------------------
    save_results([metrics])                                    # FILTER 14

    print("\n" + "="*60)
    print("TRAINING COMPLETED SUCCESSFULLY")
    print("="*60)
    print("Model is ready for deployment!")



# ============================================================
# Entry point
# ============================================================
if __name__ == '__main__':
    run_pipeline(data_path='loan_data.csv')
```

9.2 app.py — Query Side (Streamlit Prediction Application)

```
"""
╔══════════════════════════════════════════════════════════════════╗
║          CQRS PATTERN — QUERY SIDE                               ║
  Responsibility : READ from the model store + predict             ║
  Triggers       : Every user interaction in the browser           ║
  Input          : *.joblib artifacts written by Command side      ║
╚══════════════════════════════════════════════════════════════════╝

CQRS Separation
---------------
  COMMAND (train_loan_model.py)  │  QUERY (this file)
  ─────────────────────────────────┼──────────────────────────────────
  Trains the model               │  Loads trained artifacts
  Encodes & scales features      │  Encodes & scales user input
  WRITES *.joblib to disk        │  READS *.joblib from disk
  WRITES CSV reports             │  READS CSV reports
  No HTTP / UI concerns          │  No training logic
  Run once (or on demand)        │  Runs on every user request

Query flow inside this file:
  1. QUERY STORE   — load artifacts from disk (cached)
  2. QUERY PARAMS  — collect user inputs (sidebar)
  3. QUERY PROCESSOR — preprocess uploaded / default data
  4. QUERY EXECUTION — model.predict() → predictions
  5. QUERY RESULT  — render metrics, confusion matrix, table
"""

import streamlit as st
import pandas as pd
import numpy as np
import joblib
import os
from sklearn.metrics import (
    confusion_matrix, classification_report, accuracy_score,
    precision_score, recall_score, f1_score, roc_auc_score, matthews_corrcoef
)
import plotly.figure_factory as ff
import plotly.graph_objects as go
import warnings
warnings.filterwarnings('ignore')

# Set page config
st.set_page_config(
    page_title="Loan Status Classification",
    page_icon="💰",
    layout="wide",
    initial_sidebar_state="expanded"
)

# Custom styling
st.markdown("""
    <style>
    .metric-box {
        padding: 20px;
        border-radius: 10px;
        background-color: #f0f2f6;
        margin: 10px 0;
    }
    .header-title {
        color: #1f77b4;

        font-size: 2.5em;
        font-weight: bold;
        margin-bottom: 10px;
    }
    </style>
""", unsafe_allow_html=True)

# Title and description
st.markdown("<div class='header-title'>💰 Loan Status Classification Model</div>", unsafe_allow_html=True)
st.markdown("""
This interactive application demonstrates **ML classification model** trained to predict
loan approval status. Explore model performance, upload test data, and generate predictions.
""")

st.divider()
```

```
# ─────────────────────────────────────────────────────────────────────────────
# CQRS — QUERY STORE
# Read-only access to artifacts produced by the Command side.
# All loaders are cached so the store is only read once per session.
# This file NEVER writes to these artifacts — strict CQRS boundary.
# ─────────────────────────────────────────────────────────────────────────────

@st.cache_resource
def load_models():
    models = {
        'Logistic Regression': joblib.load('model/logistic_regression.joblib')
    }
    return models

@st.cache_resource
def load_preprocessors():
    scaler = joblib.load('model/scaler.joblib')
    label_encoders = joblib.load('model/label_encoders.joblib')
    feature_names = joblib.load('model/feature_names.joblib')
    return scaler, label_encoders, feature_names

@st.cache_data
def load_test_data():
    return pd.read_csv('model/test_data.csv')

@st.cache_data
def load_results():
    return pd.read_csv('model/model_results.csv')

# Load all resources
models = load_models()
scaler, label_encoders, feature_names = load_preprocessors()
test_data = load_test_data()
results_df = load_results()

# ─────────────────────────────────────────────────────────────────────────────
# CQRS — QUERY PARAMETERS
# Collect all user-supplied inputs that shape what the query returns.
# Nothing is written to disk here; selections only affect this session.
# ─────────────────────────────────────────────────────────────────────────────

st.sidebar.header("📊 Configuration")

# Section 0: Test Data Download

st.sidebar.subheader("Test Data Download")
st.sidebar.download_button(
    label="📥 Download Test Data (CSV)",
    data=test_data.to_csv(index=False),
    file_name="test_data.csv",
    mime="text/csv",
    help="Download test data to verify model predictions"
)
st.sidebar.info("💾 Download test data and upload back to app to verify model predictions")

st.sidebar.divider()

# Section 1: Model Selection
st.sidebar.subheader("Model Selection")
selected_model = st.sidebar.selectbox(
    "Select a classification model:",
    list(models.keys()),
    help="Choose the ML model to use for predictions"
)

st.sidebar.divider()

# Section 2: Data Upload
st.sidebar.subheader("Data Upload")
st.sidebar.info("💡 Upload CSV file with loan features (optional). If not uploaded, test data will be used.")

uploaded_file = st.sidebar.file_uploader(
    "Upload CSV file for predictions",
    type="csv",
    help="CSV should contain the same features as training data"
)

if uploaded_file is not None:
    try:
        user_data = pd.read_csv(uploaded_file)

        # Check if loan_status exists for confusion matrix
```

```
        user_loan_status = None
        if 'loan_status' in user_data.columns:
            user_loan_status = user_data['loan_status'].values
            user_data = user_data.drop('loan_status', axis=1)

        data_to_use = user_data
        use_user_data = True
        has_user_labels = user_loan_status is not None
        st.sidebar.success(f"✓ Loaded {len(user_data)} records")
    except Exception as e:
        st.sidebar.error(f"Error loading file: {str(e)}")
        data_to_use = test_data.drop('loan_status', axis=1)
        use_user_data = False
        has_user_labels = False
else:
    data_to_use = test_data.drop('loan_status', axis=1)
    use_user_data = False
    has_user_labels = False

st.sidebar.divider()

# ─────────────────────────────────────────────────────────────────────────────
# CQRS — QUERY PROCESSOR

# Transforms raw user input into the format the model expects.
# Uses the same encoders/scaler fitted by the Command side — read-only.
# ─────────────────────────────────────────────────────────────────────────────

@st.cache_data
def preprocess_data(data, _encoders, _scaler, feature_list):
    """Apply label encoding and scaling to data"""
    data_processed = data.copy()

    # Apply label encoders
    for col in _encoders.keys():
        if col in data_processed.columns:
            try:
                data_processed[col] = _encoders[col].transform(data_processed[col].astype(str))
            except Exception as e:
                st.error(f"Error encoding {col}: {str(e)}")
                return None

    # Select only feature columns in correct order
    X = data_processed[feature_list].copy()

    # Apply scaler
    try:
        X_scaled = _scaler.transform(X)
        X_scaled_df = pd.DataFrame(X_scaled, columns=feature_list)
        return X_scaled_df
    except Exception as e:
        st.error(f"Error scaling data: {str(e)}")
        return None

# Preprocess the data
data_preprocessed = preprocess_data(data_to_use, label_encoders, scaler, feature_names)

if data_preprocessed is None:
    st.error("Failed to preprocess data. Please check your input.")
    st.stop()

st.sidebar.divider()

# Main content area - Two columns
col1, col2 = st.columns([1, 1.2])

with col1:
    st.subheader("📈 Model Performance Metrics")

    # Get metrics for selected model
    model_row = results_df[results_df['Model'] == selected_model].iloc[0]

    # Display metrics in grid
    metric_col1, metric_col2, metric_col3 = st.columns(3)

    with metric_col1:
        st.metric(
            "Accuracy",
            f"{model_row['Accuracy']:.4f}",
            help="Percentage of correct predictions"
        )
        st.metric(
```

```
            "Precision",
            f"{model_row['Precision']:.4f}",

            help="True positives / All positive predictions"
        )

    with metric_col2:
        st.metric(
            "AUC-ROC",
            f"{model_row['AUC']:.4f}",
            help="Area under ROC curve (0-1, higher is better)"
        )
        st.metric(
            "Recall",
            f"{model_row['Recall']:.4f}",
            help="True positives / All actual positives"
        )

    with metric_col3:
        st.metric(
            "F1 Score",
            f"{model_row['F1']:.4f}",
            help="Harmonic mean of precision and recall"
        )
        st.metric(
            "MCC",
            f"{model_row['MCC']:.4f}",
            help="Matthews Correlation Coefficient (-1 to 1)"
        )

with col2:
    st.subheader("🏆 Model Evaluation")

    # Create comparison chart
    comparison_data = results_df[['Model', 'Accuracy', 'AUC', 'F1']].copy()
    comparison_data = comparison_data.sort_values('Accuracy', ascending=True)

    fig = go.Figure()
    fig.add_trace(go.Bar(name='Accuracy', y=comparison_data['Accuracy'], x=comparison_data['Model'], orientation='v'))
    fig.add_trace(go.Bar(name='AUC', y=comparison_data['AUC'], x=comparison_data['Model'], orientation='v'))
    fig.add_trace(go.Bar(name='F1', y=comparison_data['F1'], x=comparison_data['Model'], orientation='v'))

    fig.update_layout(
        height=400,
        barmode='group',
        showlegend=True,
        margin=dict(l=150, r=50, t=50, b=50)
    )
    st.plotly_chart(fig, use_container_width=True)

st.divider()

# ──────────────────────────────────────────────────────────────────────────────
# CQRS — QUERY EXECUTION
# The single predict() call is the Query result boundary.
# model.predict() is stateless and read-only — CQRS Query handler.
# ──────────────────────────────────────────────────────────────────────────────

st.subheader("🔮 Model Predictions on Test Data")

model = models[selected_model]

# Execute the Query: produce predictions from preprocessed input

y_pred = model.predict(data_preprocessed)

# Determine if we can calculate confusion matrix
should_show_cm = False
y_true = None

if not use_user_data:
    # Using default test data - always have labels
    y_true = test_data['loan_status'].values
    should_show_cm = True
elif has_user_labels:
    # User uploaded data with labels
    y_true = user_loan_status
    should_show_cm = True

if should_show_cm:
    # Calculate confusion matrix with labels in order: 0=Not Approved, 1=Approved
    cm = confusion_matrix(y_true, y_pred, labels=[0, 1])
    # cm structure: rows=[0,1], cols=[0,1]
```

```
    # Row 0: Not Approved actual [TN, FP]
    # Row 1: Approved actual [FN, TP]

    col1, col2 = st.columns([1, 1])

    with col1:
        st.subheader("Confusion Matrix")
        st.write("**Rows: Actual | Columns: Predicted**")

        # For display order: Approved (1) top, Not Approved (0) bottom
        # X-axis: Approved (1) left, Not Approved (0) right
        # We need to rearrange both rows and columns
        cm_display = cm[[1, 0], :][:, [1, 0]]
        # cm_display Row 0: Approved actual [TP, FN]
        # cm_display Row 1: Not Approved actual [FP, TN]

        # Create heatmap
        # Plotly will display: first array element at bottom, last at top
        # So reverse the data array to match reversed y-labels
        cm_for_plot = cm_display[::-1]  # Flip rows for plotly display

        fig_cm = ff.create_annotated_heatmap(
            z=cm_for_plot,
            x=['Approved (1)', 'Not Approved (0)'],
            y=['Not Approved (0)', 'Approved (1)'],
            colorscale='Blues',
            showscale=True,
            text=cm_for_plot,
            texttemplate='%{text}'
        )
        fig_cm.update_layout(title_text='', height=400, width=400)
        fig_cm.update_xaxes(title_text='Predicted Label')
        fig_cm.update_yaxes(title_text='Actual Label')

        st.plotly_chart(fig_cm, use_container_width=True)

    with col2:
        st.subheader("Classification Report")

        # Get classification report with explicit order: 0, 1
        class_report = classification_report(

            y_true, y_pred,
            labels=[0, 1],
            target_names=['Not Approved (0)', 'Approved (1)'],
            output_dict=True
        )

        report_df = pd.DataFrame(class_report).transpose().round(4)
        st.dataframe(
            report_df,
            use_container_width=True,
            column_config={
                "precision": st.column_config.NumberColumn(format="%.4f"),
                "recall": st.column_config.NumberColumn(format="%.4f"),
                "f1-score": st.column_config.NumberColumn(format="%.4f"),
                "support": st.column_config.NumberColumn(format="%.0f"),
            }
        )
else:
    st.info("📝 Upload data with 'loan_status' column to view confusion matrix and classification report")

st.divider()

# Show sample predictions
st.subheader("📋 Sample Predictions")
st.write(f"Showing first 10 predictions from {len(data_to_use)} records")

# Display original encoded values (before scaling), not scaled values
predictions_df = data_to_use.head(10).copy()
predictions_df['Prediction'] = y_pred[:10]
predictions_df['Prediction'] = predictions_df['Prediction'].map({0: 'Not Approved', 1: 'Approved'})

if not use_user_data and 'loan_status' in test_data.columns:
    predictions_df['Actual'] = test_data['loan_status'].head(10).map({0: 'Not Approved', 1: 'Approved'})

st.dataframe(predictions_df, use_container_width=True, hide_index=True)

st.divider()

# Footer
st.markdown("""
---
```

```
**Model Details:**
- **Logistic Regression**: Linear model for binary classification
Model use StandardScaler for feature normalization and LabelEncoder for categorical features.
""")
```

10. Appendix B — Training Pipeline Output Log

Console output captured from running the Command side (python3 train_loan_model.py) on the full 45,000-row dataset:

```
============================================================
LOAN APPROVAL : CQRS ARCHITECTURE - COMMAND SIDE
============================================================
Command Handler : run_pipeline()
Responsibility  : Train model and persist artifacts
Pattern         : Pipe-and-Filter
Pipeline        : Each filter processes data and passes
                  the output to the next filter.
============================================================

Starting Loan Approval Training Pipeline...

[FILTER 1] Loading dataset from 'loan_data.csv' ...
           Shape: (45000, 14)
           Target distribution:
loan_status
0    35000
1    10000
Name: count, dtype: int64

[FILTER 2] Dropping missing values ...
           Shape after drop: (45000, 14)

[FILTER 3] Removing age outliers (person_age > 120) ...
           Shape after outlier removal: (44995, 14)

[FILTER 4] Splitting features and target ('loan_status') ...
           Categorical columns : ['person_gender', 'person_education', 'person_home_ownership', 'loan_intent',
'previous_loan_defaults_on_file']
           Numerical columns   : ['person_age', 'person_income', 'person_emp_exp', 'loan_amnt', 'loan_int_rate',
'loan_percent_income', 'cb_person_cred_hist_length', 'credit_score']

[FILTER 5] Splitting into train / val / test sets ...
           Train : 36000 rows
           Val   : 4495 rows
           Test  : 4500 rows

[FILTER 6] Fitting LabelEncoders on training data only ...
           Encoded 'person_gender': {'female': 0, 'male': 1}
           Encoded 'person_education': {'Associate': 0, 'Bachelor': 1, 'Doctorate': 2, 'High School': 3, 'Master': 4}
           Encoded 'person_home_ownership': {'MORTGAGE': 0, 'OTHER': 1, 'OWN': 2, 'RENT': 3}
           Encoded 'loan_intent': {'DEBTCONSOLIDATION': 0, 'EDUCATION': 1, 'HOMEIMPROVEMENT': 2, 'MEDICAL': 3, 'PERSONAL':
4, 'VENTURE': 5}
           Encoded 'previous_loan_defaults_on_file': {'No': 0, 'Yes': 1}

[FILTER 7] Applying LabelEncoders to val and test sets ...
           Encoding applied.

[FILTER 8] Fitting StandardScaler on training data only ...
           Scaler fitted.

[FILTER 9] Applying StandardScaler to val and test sets ...
           Scaling applied.

[FILTER 10] Saving original test data to 'test_data.csv' ...
            Saved 4500 rows.

[FILTER 11] Training Logistic Regression model ...
            Model training complete.

[FILTER 12] Evaluating model on test set ...
            Accuracy  : 0.8927
            AUC       : 0.9507

            Precision : 0.7630
            Recall    : 0.7500
            F1 Score  : 0.7564
            MCC       : 0.6876

[FILTER 13] Saving model and preprocessor artifacts ...
            Saved: logistic_regression.joblib
            Saved: scaler.joblib
```

```
            Saved: label_encoders.joblib
            Saved: feature_names.joblib

[FILTER 14] Saving evaluation results to 'model_results.csv' ...

============================================================
SUMMARY RESULTS
============================================================
              Model  Accuracy      AUC  Precision  Recall      F1      MCC
Logistic Regression  0.892667 0.950729    0.76297    0.75 0.75643 0.687645

============================================================
TRAINING COMPLETED SUCCESSFULLY
============================================================
Model is ready for deployment!
```

SEML Assignment 1 Completed: